Database Development and Design (SOE201)

Lecture 3a: Hash Indexing

Dr. Wei Wang
AIAT

Learning Outcomes

- Hash-based Indexing
- Static Hashing
- Dynamic Hashing
  - Query
  - Insertion
- Comparison of Ordered Indexing and Hash-based Indexing

Hashing

- Bucket is unit of storage containing one or more records (bucket is typically disk block).
- Hash function *h* is function from set of all search-key values *K* to set of all bucket addresses *B*.
  - Hash function can be used to locate records for access, insertion as well as deletion.
- Records with different search-key values may be mapped to same bucket
  - Thus, entire bucket has to be searched sequentially to locate record.
  - In hash index, buckets store entries with pointers to records.
  - In hash file organisation, buckets store records.

Hash Functions

- Worst hash function maps all search-key values to same bucket; this makes access time proportional to number of search-key values in file.
- Ideal hash function is uniform, i.e. each bucket is assigned same (more or less) number of search-key values from set of all possible values.
- Ideal hash function is also random, i.e. it does not depend on actual distribution of search-key values.
- With N buckets (0, 1, 2, …, N-1), hash function h of following form works well in practice.
  - h(value) = (a*value + b) mod N

Example of Hash File Organisation

- Hash file organisation of *instructor* file, using *dept_name* as key; assume there are 8 buckets
- Binary representation of *i* th character is assumed to be integer *I*
- Hash function returns "sum of binary representations of characters modulo 8"
  - e.g. h(Music) = 1; h(History) = 2; h(Physics) = 3; h(Elec. Eng.) = 3
  - h(Music) = (13+21+19+9+3) mod 8 = 65 mod 8 = 1

A	B	C	D	E	F	G	H	I	G	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	25	25	26

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Example of Hash File Organisation – cont'd

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

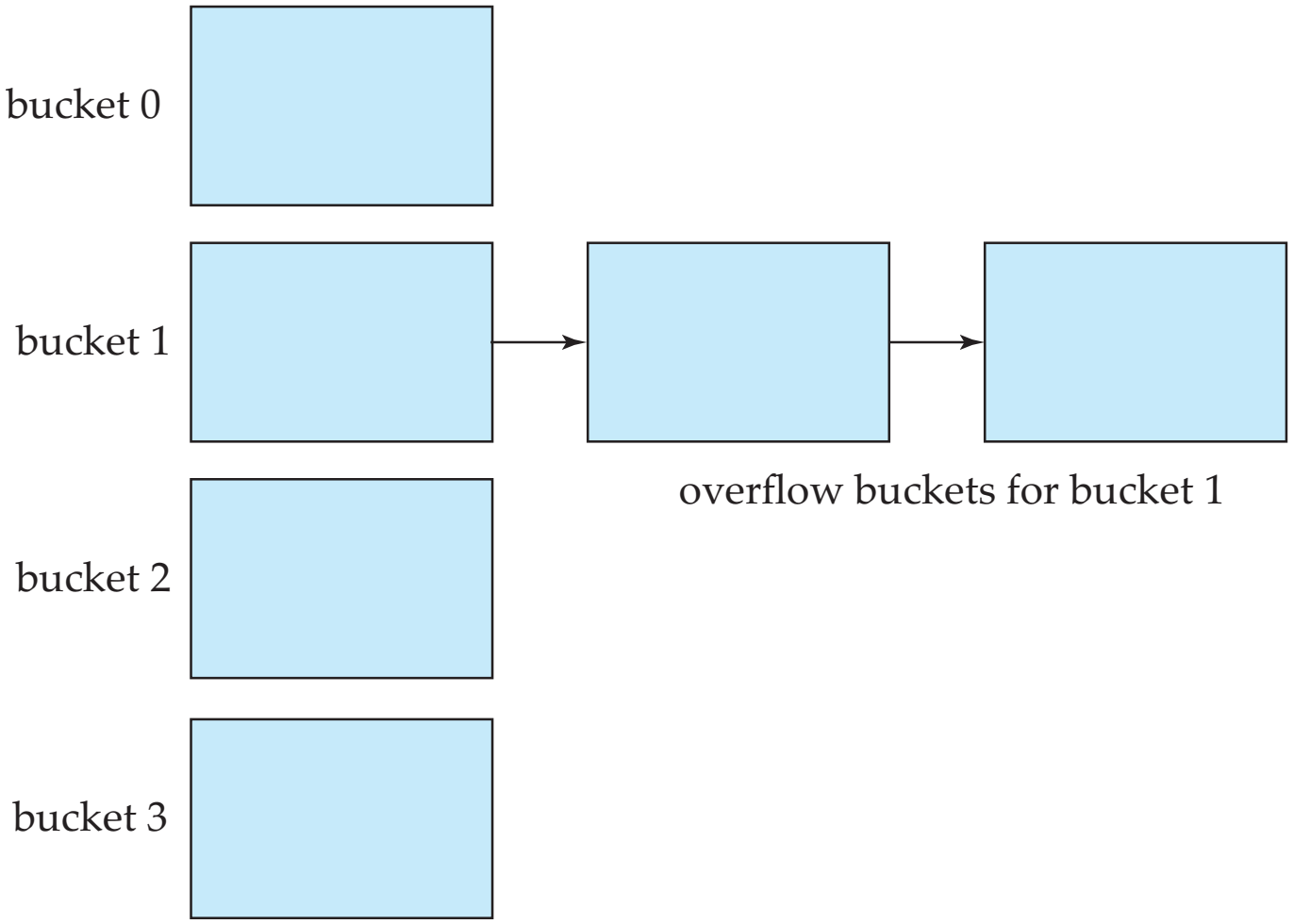
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

Handling of Bucket Overflows

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records
    - Multiple records have same search-key value
    - Chosen hash function produces non-uniform distribution of key values
- Although probability of bucket overflow can be reduced, it cannot be eliminated
  - It is handled by using overflow buckets.
  - Overflow chaining – overflow buckets of given bucket are chained together in linked list.

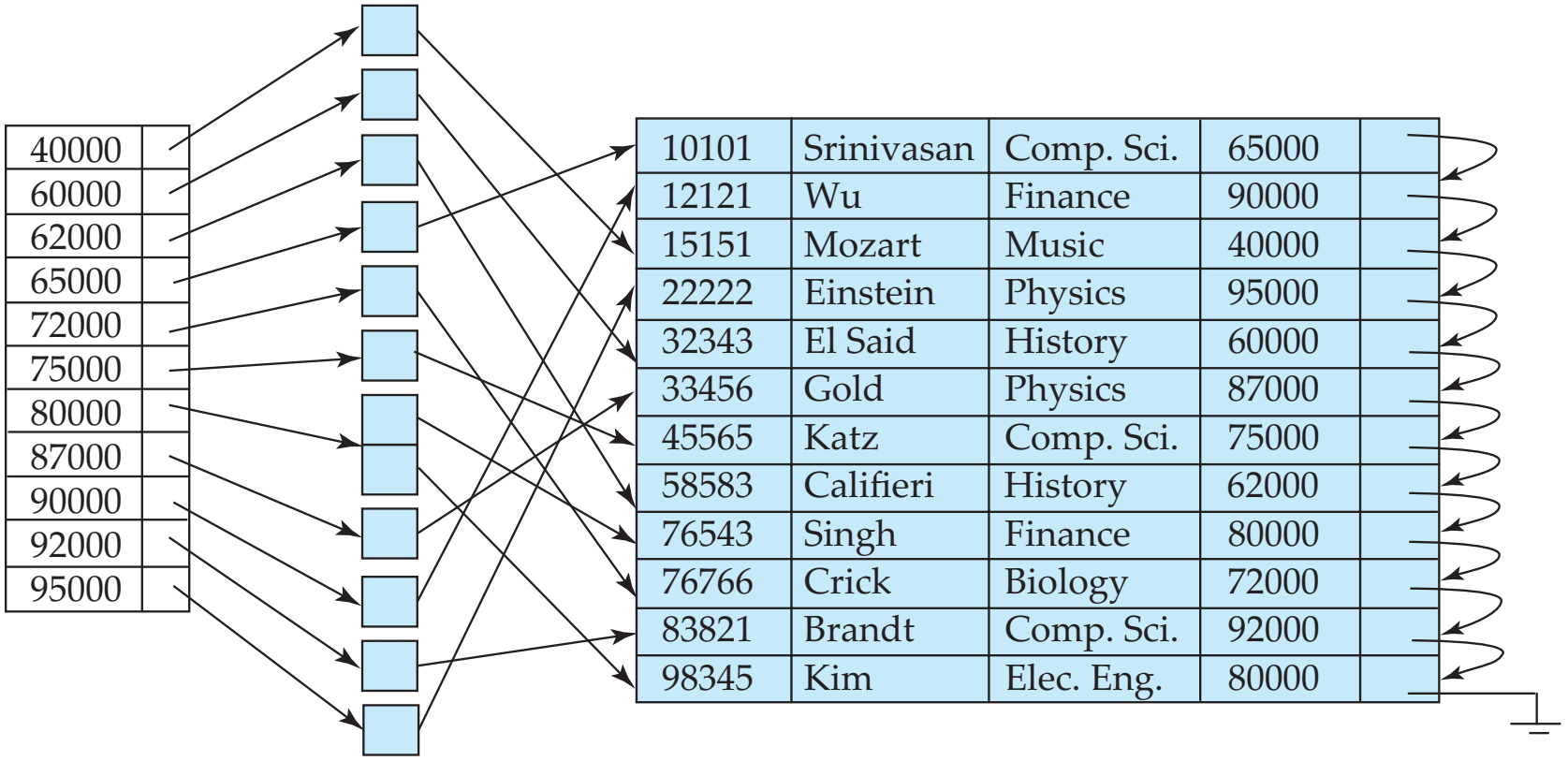
Bucket Overflows



Hash Index

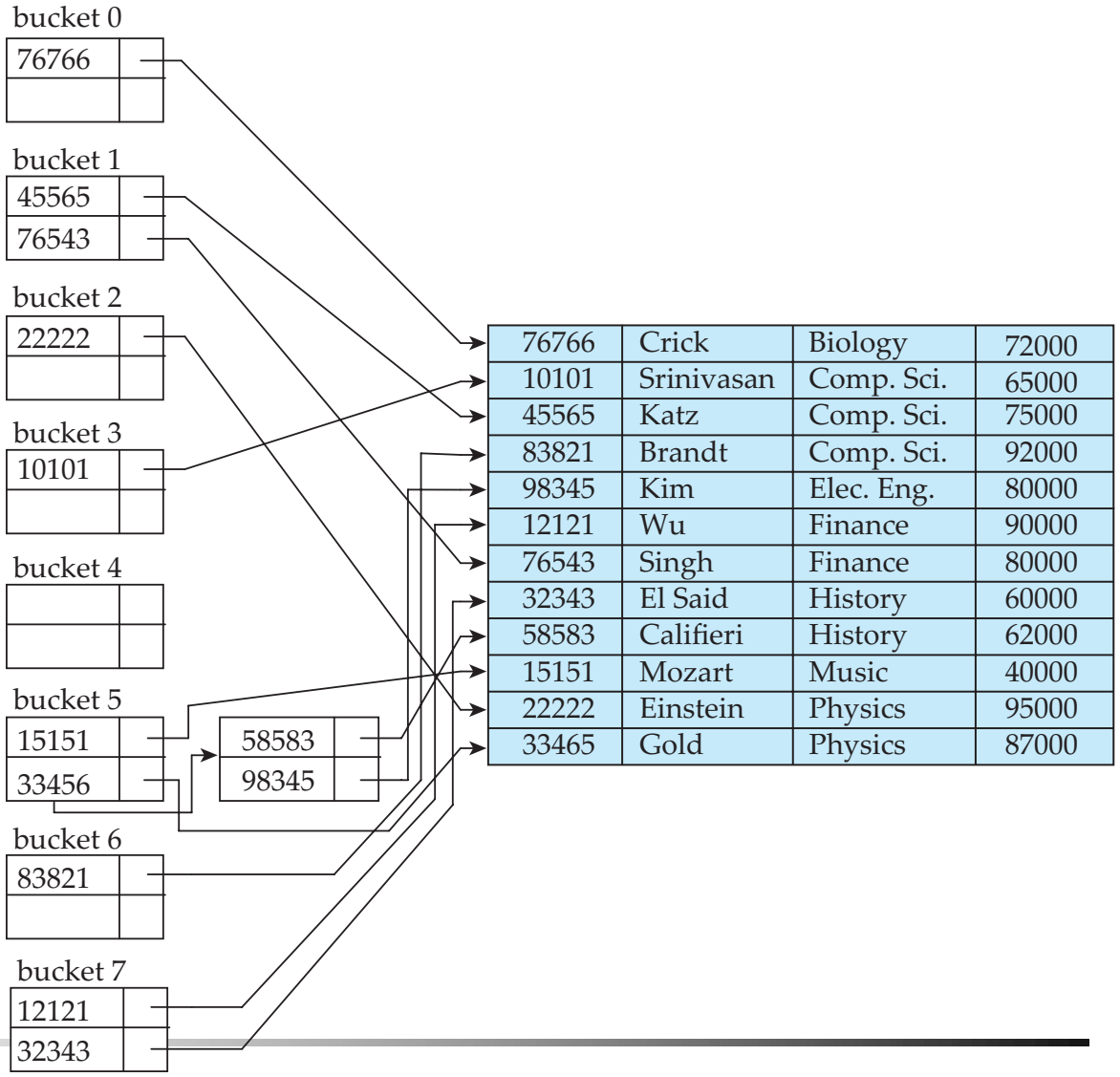
- Hashing can be used not only for file organisation, but also index-structure creation.
- Hash index organises search keys, with their associated record pointers, into hash file structure.
- Strictly speaking, hash indices are always secondary indices.

Secondary Index Example



Example of Static Hash Index

- Hash index on attribute *ID* of *instructor*
- Hash function *h* computes sum of digits in ID modulo by 8
  - e.g. *h*(15151)=13 mod 8=5.
- Bucket 5 contains many different search keys
  - Overflow chaining for bucket 5



Deficiencies of Static Hashing

- In static hashing, function *h* maps search-key values to **fixed set of *B*** bucket addresses.
  - Databases grow or shrink with time.
  - If initial number of buckets is too small, and file grows, performance will degrade due to too many overflows.
  - If space is allocated for anticipated growth, significant amount of space will be wasted initially (buckets are underfull).
    - If database shrinks, again space will be wasted.
- Solution: periodic re-organisation of file with new hash function
  - Expensive, disrupts normal operations
- Better solution: allow number of buckets to be modified dynamically - Dynamic Hashing!

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Dynamic Hashing

- Good for database that grows and shrinks in size
- Allows hash function to be modified dynamically
- Extendable hashing – one form of dynamic hashing
  - Hash function generates values over large range, typically *b*-bit integers, e.g. 32, 64 or 128.
  - At any time use only prefix of hash values to index into table of bucket addresses.
  - Let length of prefix be *i* bits, e.g. $0 \leq i \leq 32$.
    - Bucket address table size = 2^i, initially i = 0.
    - Value of i grows and shrinks as size of database grows and shrinks.
  - Multiple entries in bucket address table may point to same bucket.
  - Thus, actual number of buckets is $< 2^i$
    - Number of buckets also changes dynamically due to coalescing and splitting of buckets.

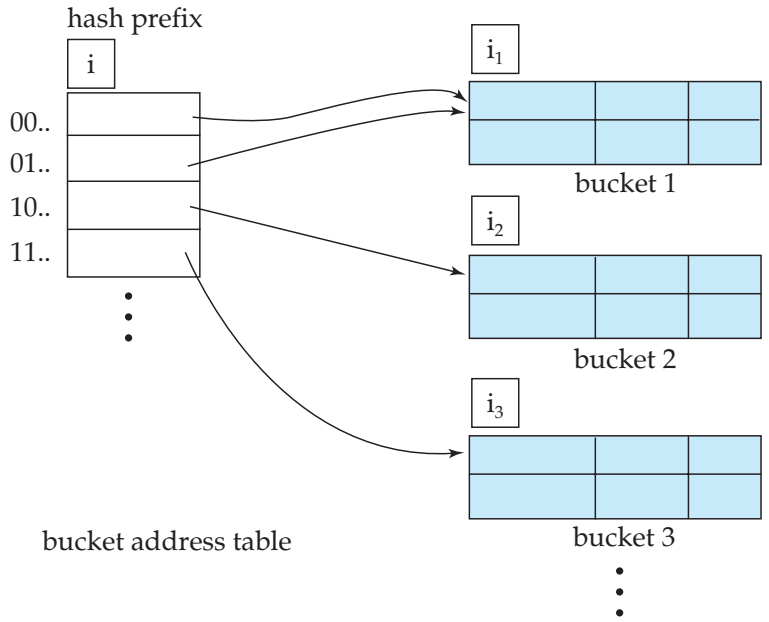
Example of Binary Representation

- i=3
  - 000, 001, 010, 011, 100, 101, 110, 111
- i=4
  - 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111

1 =	1 =	1
10 =	2+0 =	2
11 =	2+1 =	3
100 =	4+0+0 =	4
101 =	4+0+1 =	5
110 =	4+2+0 =	6
111 =	4+2+1 =	7
1000 =	8+0+0+0 =	8
1001 =	8+0+0+1 =	9
1010 =	8+0+2+0 =	10
1011 =	8+0+2+1 =	11
1100 =	8+4+0+0 =	12
1101 =	8+4+0+1 =	13
1110 =	8+4+2+0 =	14
1111 =	8+4+2+1 =	15
10000 =	16+0+0+0+0 =	16
10001 =	16+0+0+0+1 =	17
10010 =	16+0+0+2+0 =	18
10011 =	16+0+0+2+1 =	19
10100 =	16+0+4+0+0 =	20
10101 =	16+0+4+0+1 =	21
10110 =	16+0+4+2+0 =	22
10111 =	16+0+4+2+1 =	23
11000 =	16+8+0+0+0 =	24
11001 =	16+8+0+0+1 =	25
11010 =	16+8+0+2+0 =	26
11011 =	16+8+0+2+1 =	27
11100 =	16+8+4+0+0 =	28
11101 =	16+8+4+0+1 =	29
11110 =	16+8+4+2+0 =	30
11111 =	16+8+4+2+1 =	31

Extendable Hash Index Structure

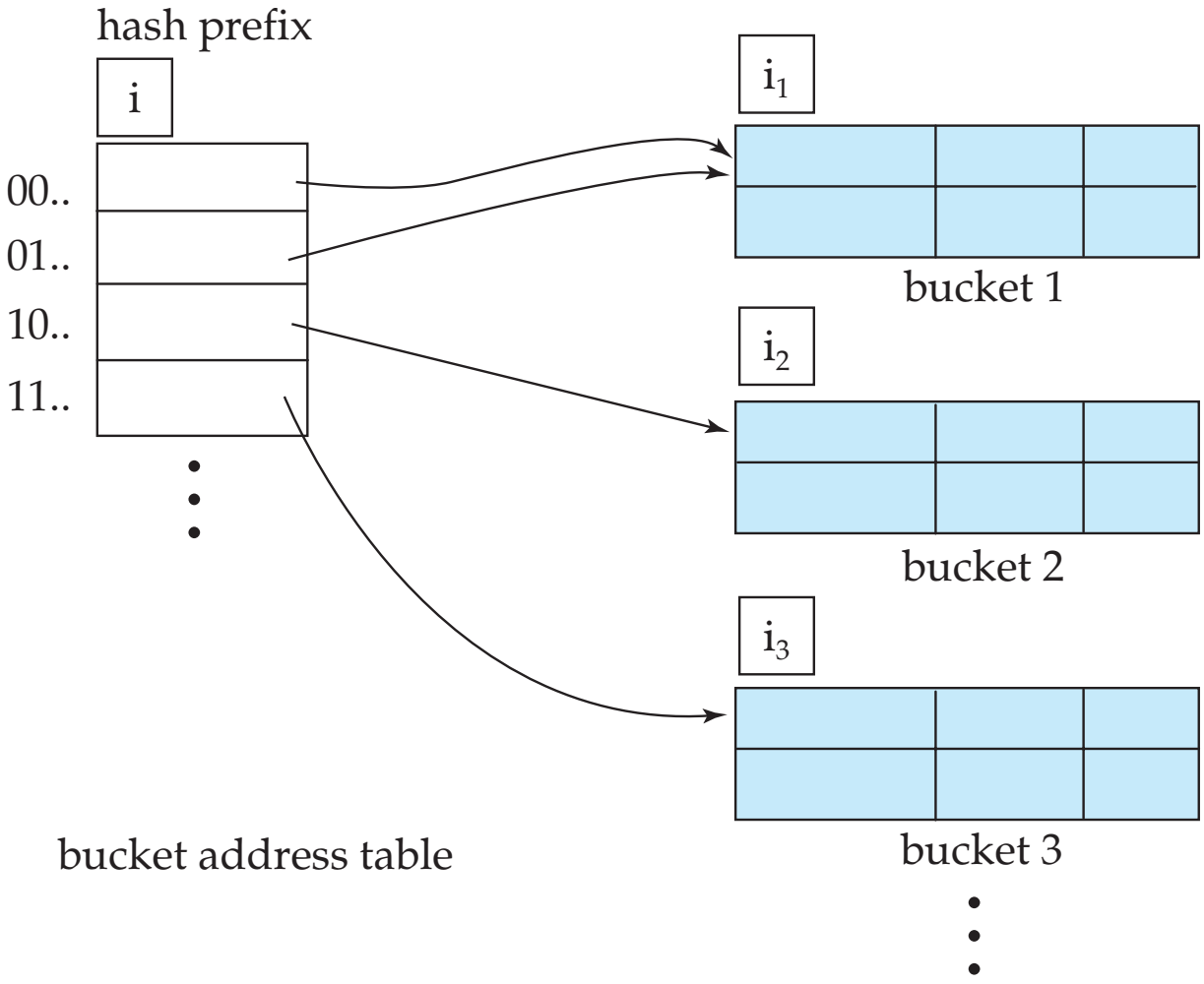
- Let length of prefix be *i* bits (write it on top left of bucket address table)
- Each bucket *j* stores a value i_j (write it on top left of bucket)



- All entries in bucket address table that point to same bucket have same hash values on first i_j bits.
- Number of bucket address table entries that point to bucket *j* is:

$$2^{(i - i_j)}$$

Extendable Hash Index Structure – cont'd



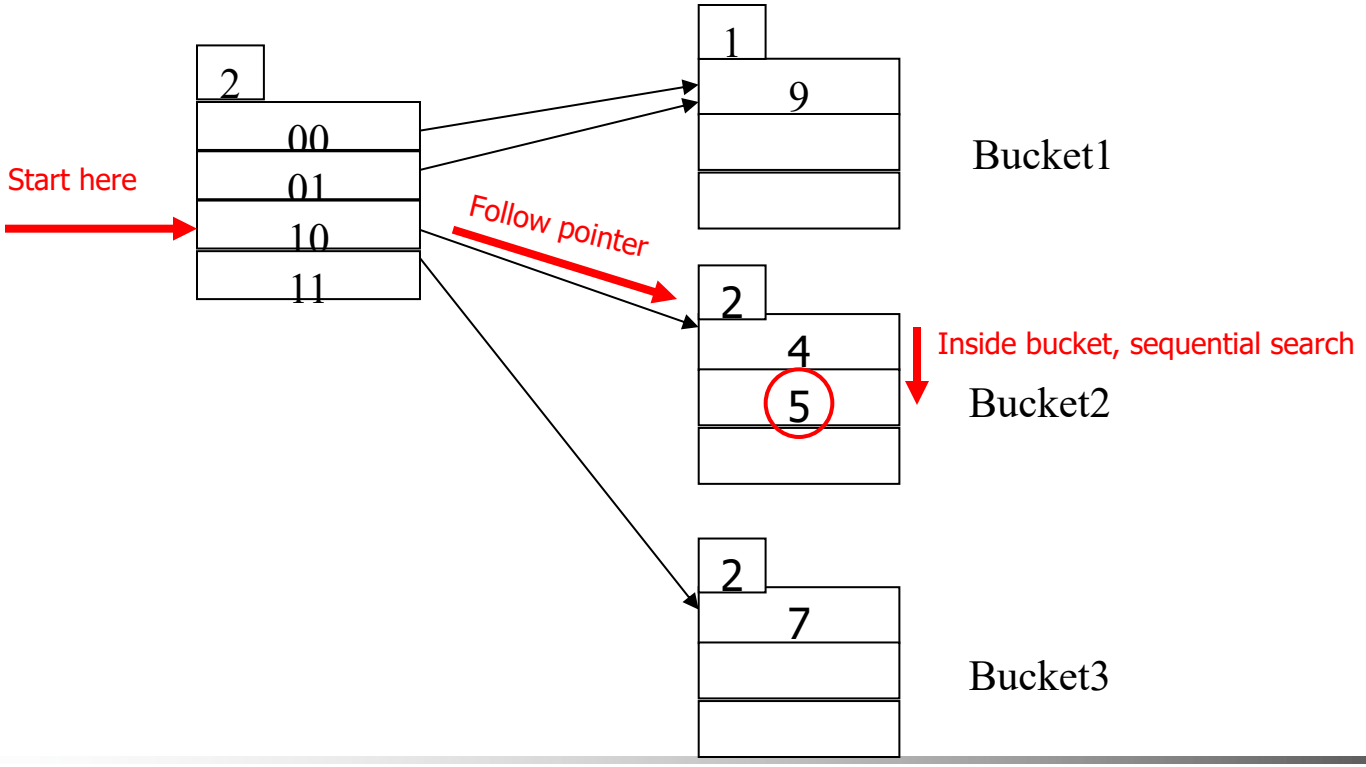
In this structure, $i = 2$, $i_1 = 1$, $i_2 = i_3 = i$

Queries

- To locate bucket containing search-key K_j:
  - 1. Compute $h(K_j) = X$
  - 2. Use first i high order bits of X as displacement into bucket address table, and follow pointer to appropriate bucket.

Query Example

- Assume $h(x) = x \bmod 8$; look up search key 5
- $h(5) = 5 \bmod 8 = 5 = 101$
- i=2; only look at first two bits (from left), 101



Insertion

- To insert *record* with search-key value K_j
  - Follow same procedure as look-up and locate bucket, say j.
  - If there is room in bucket j, insert record in bucket.
  - Else, bucket must be split and insertion re-attempted (next slide)
    - Overflow buckets used instead in some cases (will see shortly)

Insertion cont'd

- To split bucket j, when inserting record with search-key value K_j:
  - If $i > i_j$ (*more than one pointer pointing to bucket j*)
    - Allocate new bucket z, and set $i_j = i_z = (i_j + 1)$
    - Update second half of bucket address table entries originally pointing to j, to point to z
    - Remove each record in bucket j and reinsert (possibly in j or z)
    - Re-compute new bucket for K_j and insert record in bucket
      - *Further splitting is required if the bucket is still full*

Insertion cont'd

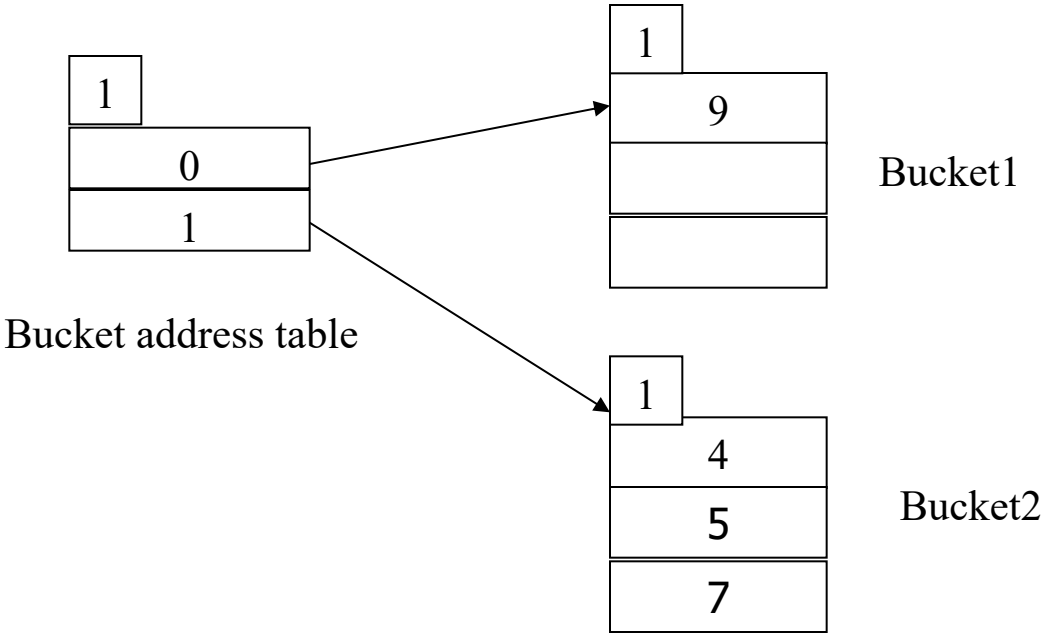
- If $i = i_j$ (*only one pointer to bucket j*)
  - *If i reaches some limit b, or too many splits have happened but still cannot insert, revert to original index and create **overflow** bucket.*
  - Else
    - Increment *i* and double size of bucket address table;
    - Replace each entry in bucket address table by two entries that point to same bucket;
    - Now $i > i_j$ so use first case above (i.e. $i > i_j$).

Deletion

- To delete key value
  - Locate it in its bucket and remove it.
  - Bucket itself can be removed if it becomes empty (with appropriate updates to bucket address table).
  - Coalescing of buckets can be done (can coalesce only with "buddy" bucket having same value of i_j and same i_j –1 prefix, if present)
  - Decreasing bucket address table size is also possible
    - Note: decreasing bucket address table size is expensive and should be done only if number of buckets becomes much smaller than bucket address table

Insertion Example

- Consider extendable hashing with hash function $h(x) = x \bmod 8$ and one bucket can hold three records.
- Initial hash index shown below. Draw hash index after inserting record with search key 13.

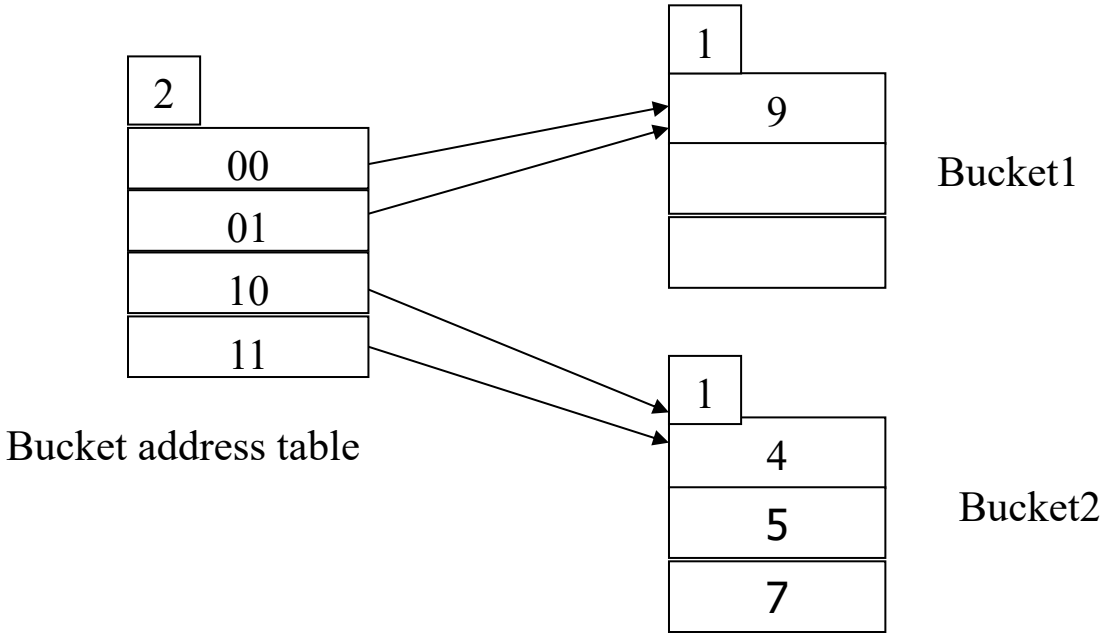


Insertion Example – cont'd

- h(13) = 13 mod 8 = 5 = 101,
- i=1, only look at first bit from left
- 13 (101) should be in bucket 2, but there is no space and bucket 2 also can't be split so we have to extend hash index structure
- Double size of bucket address table, i.e. i=2
- Rearrange pointers (pointers to each bucket doubled)
- Now $i > i_j$, $i = 2$, $i_j = 1$
- Split bucket 2 by creating new bucket, and redistribute entries
- Re-insert all search keys in bucket 2
- Re-insert search key 13

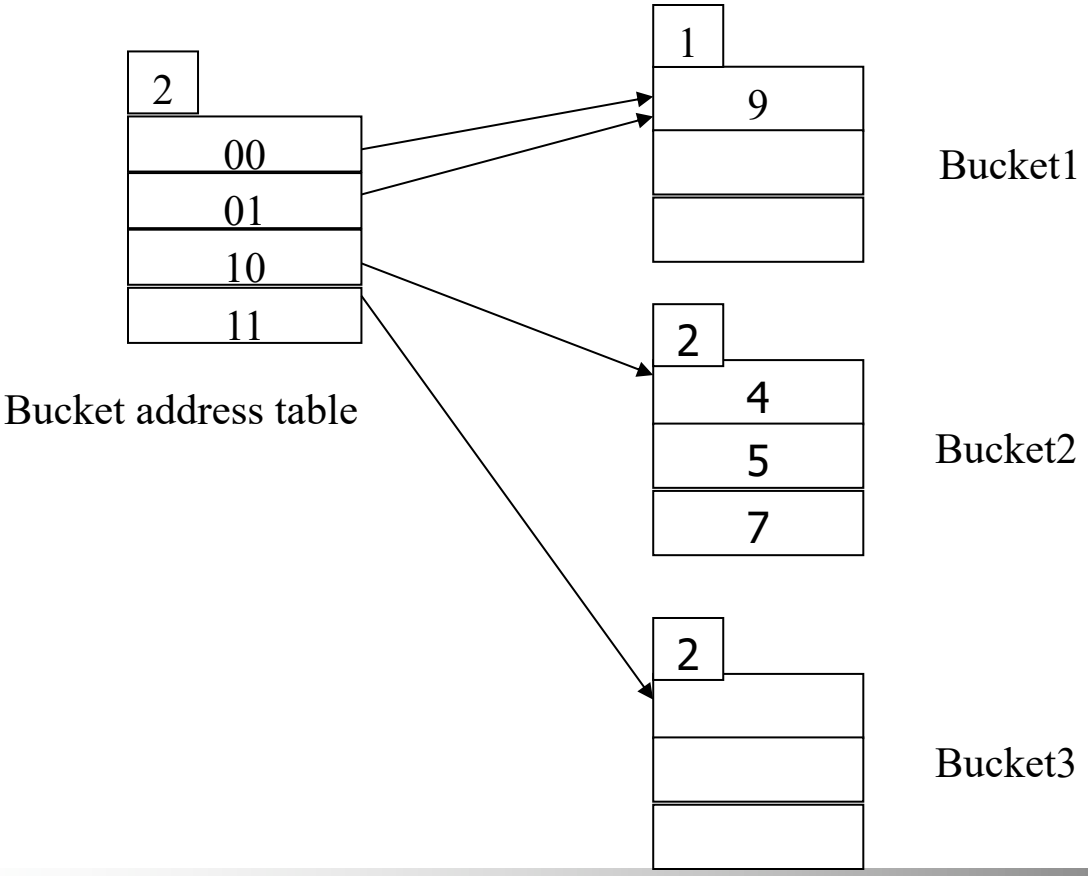
Double Bucket Address Table

- **Intermediate** hash index is shown below after doubling size of the bucket address table.
  - Search key 13 should be inserted to Bucket 2.



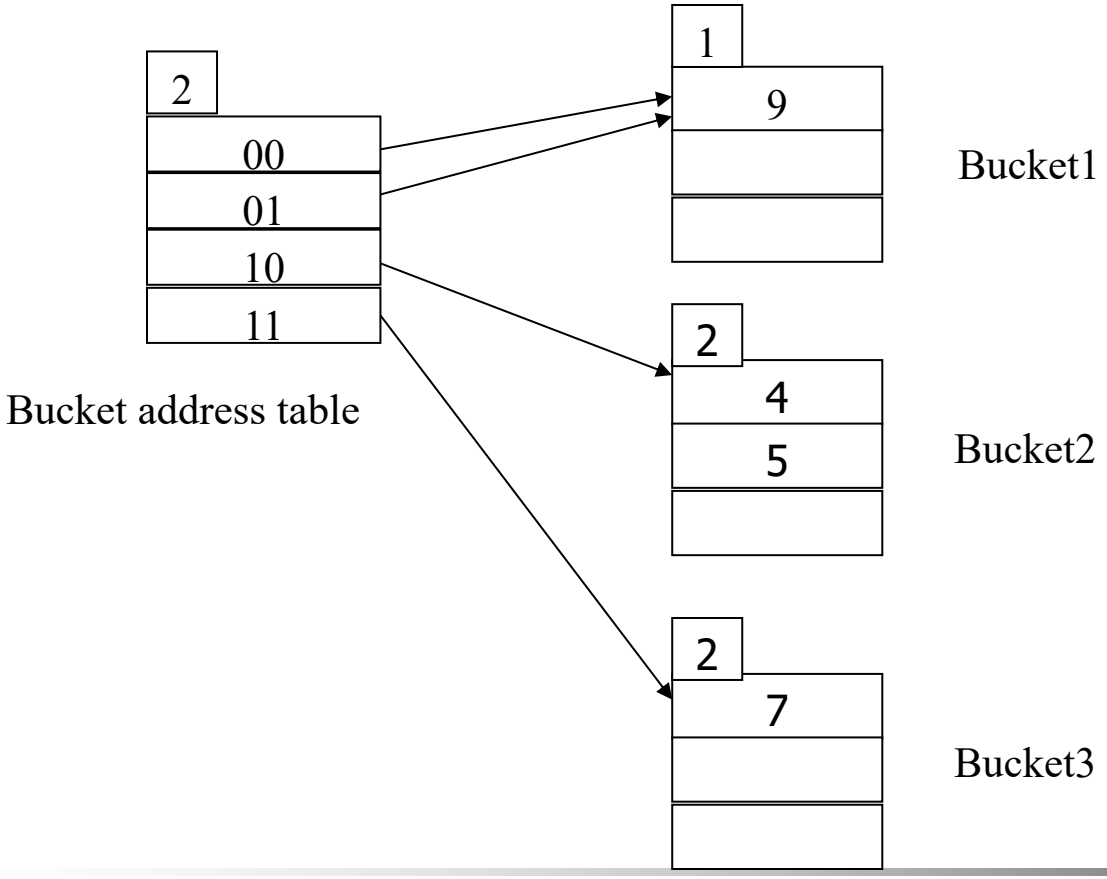
Split Bucket 2

- **Intermediate** hash index is shown below
- Split original "bucket 2".
- Increment i_j and i_z for Bucket 2 and 3.



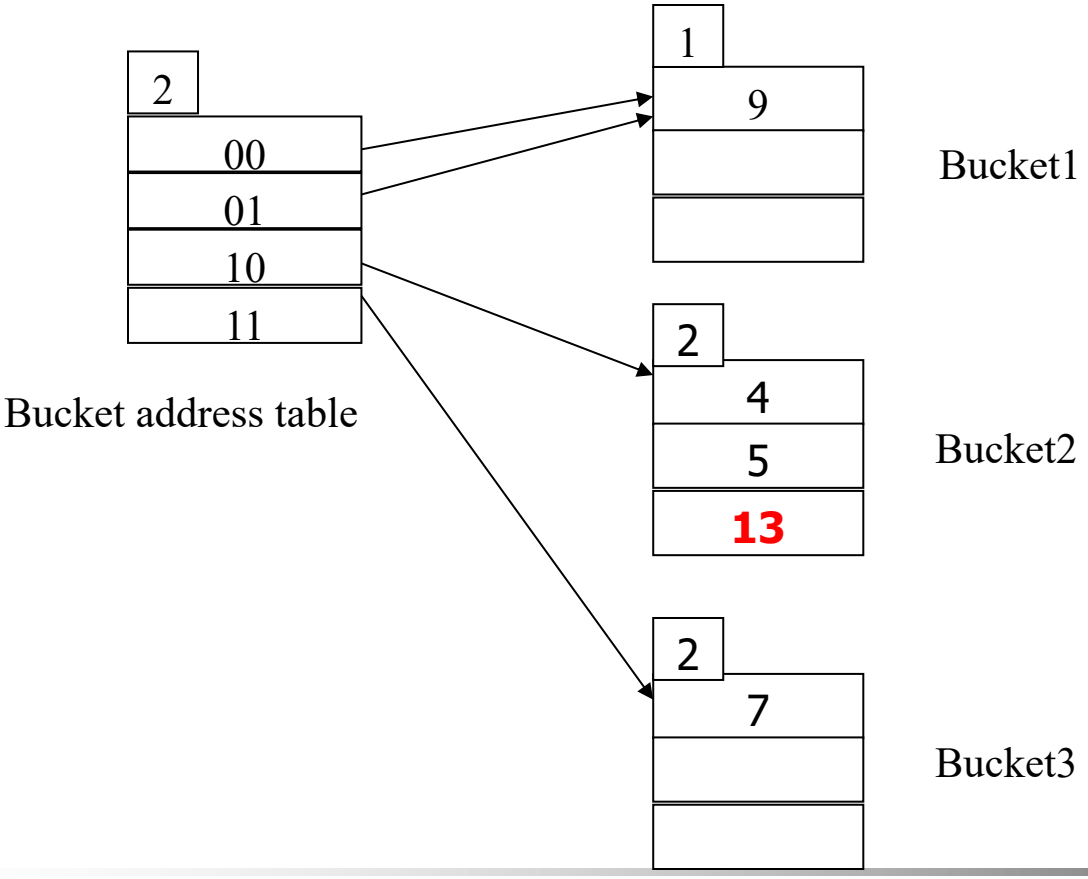
Re-insert Search Keys

- **Intermediate** hash index is shown below
- Reinsert search keys 4 (100), 5 (101) and 7 (111).



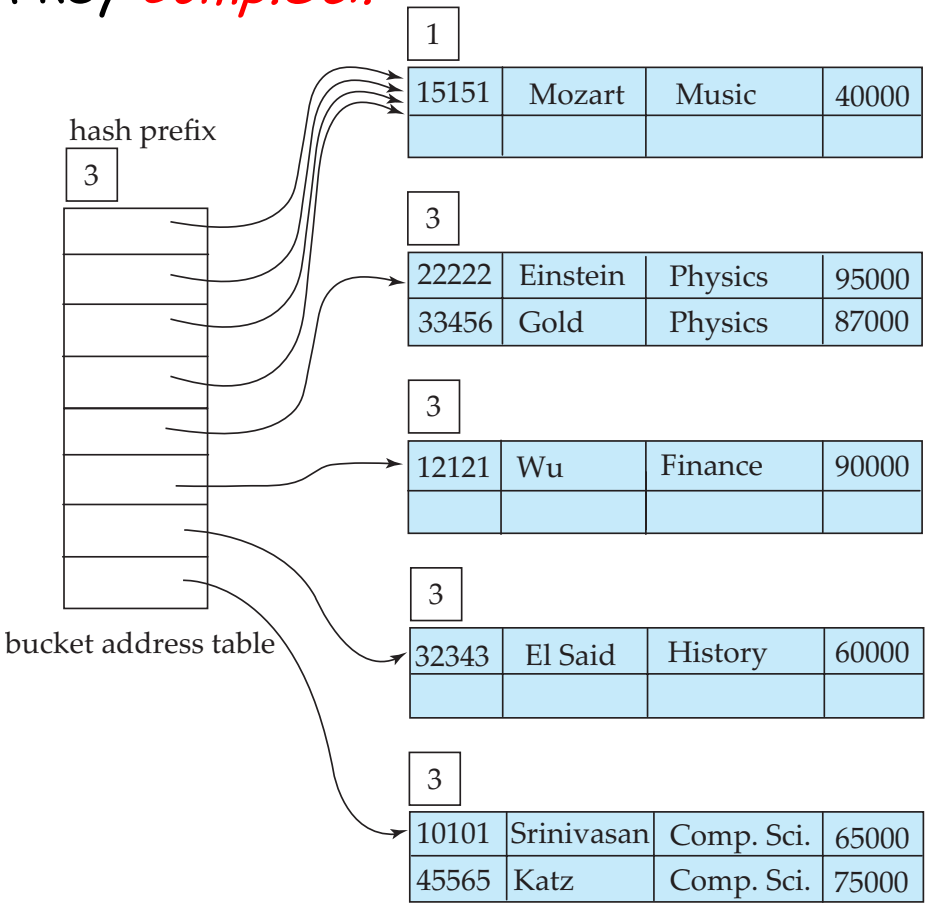
Final Result

- Re-insert search key 13.



Another Example

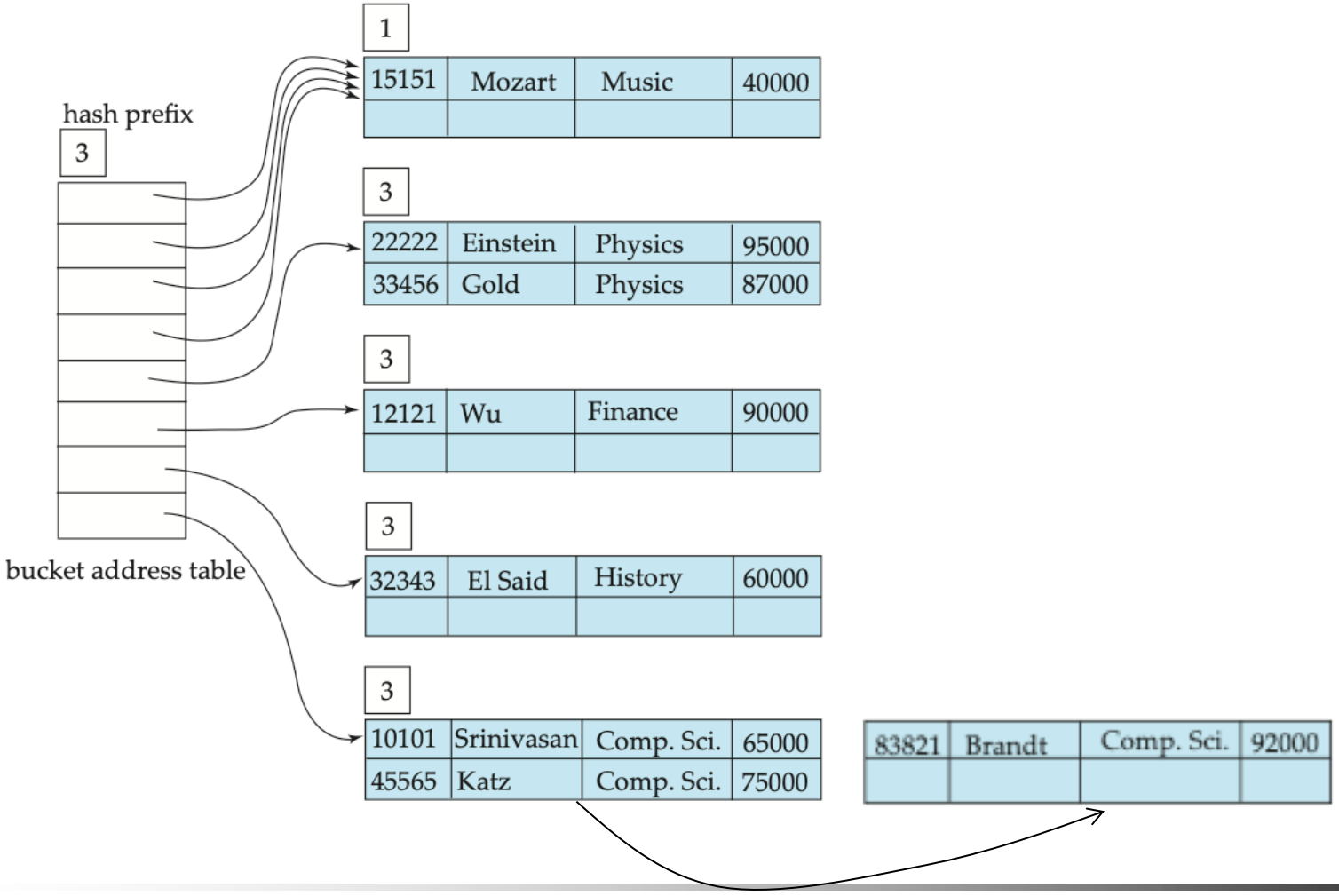
- Assume hash index built on "Department"
- Insert another record "*83821, Brandt, Comp.Sci., 92000*" with search key *Comp.Sci.*



*If i reaches some limit b, or too many splits have happened but still cannot insert, revert to original index and create **overflow** bucket.*

Another Example

- *Use overflow bucket*



Note on Insertion

- One split might not solve problem
- Multiple split also might not solve problem
- But split cannot go infinitely
- Add conditions to stop split
  - If i reaches some limit b, or too many splits have happened but still cannot insert (will specify how many times in questions)
  - Then *revert* to original index (before first split) and create *overflow* bucket.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Case Study

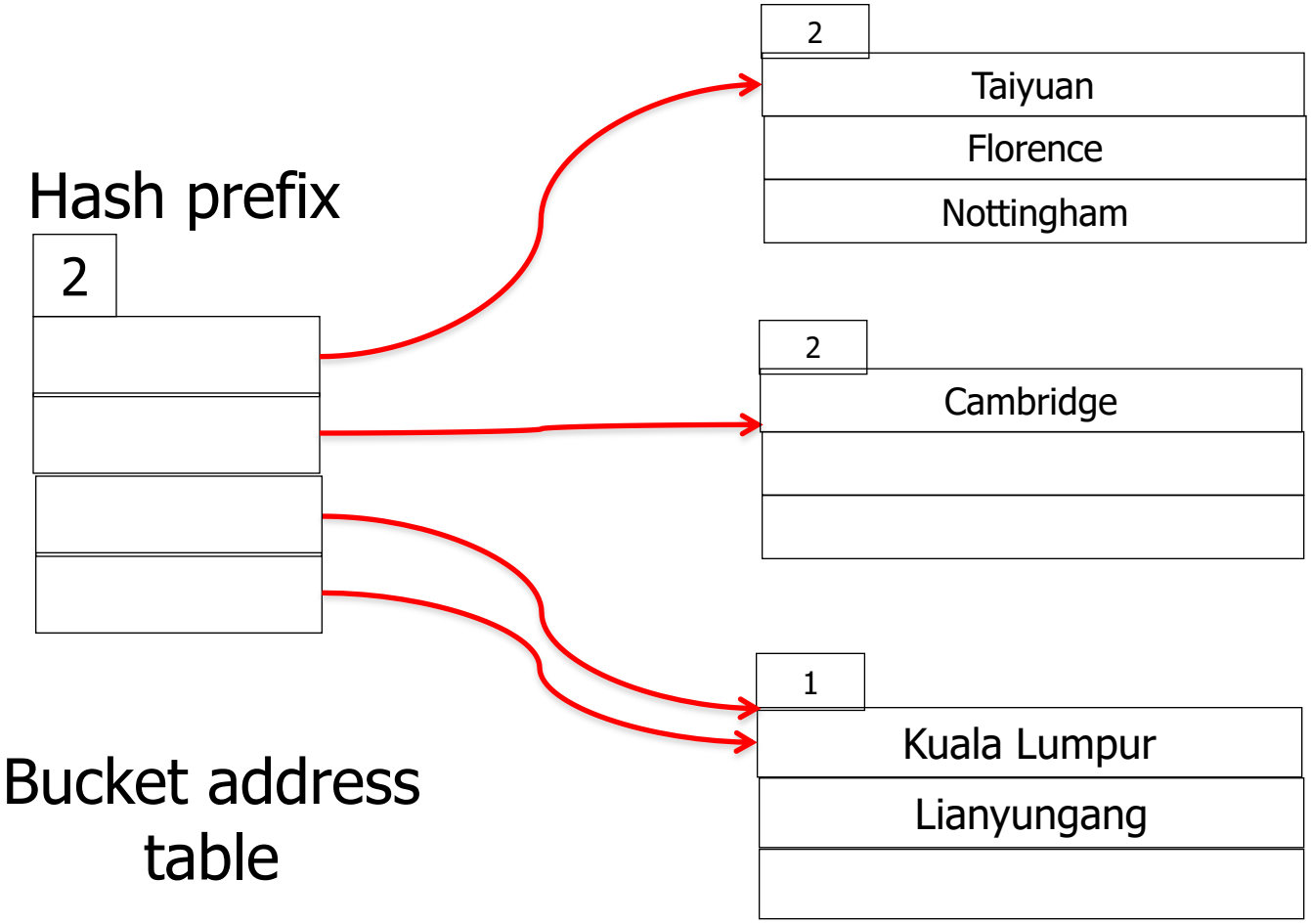
- Assume that the initial extendable hash index on city names is shown below, and one block can hold up to three records.
- Based on the city names and their hash values (already computed based on the hash function) in the following table, draw hash indices after inserting some new records.
- In insertion, overflow buckets should be used, if after *TWO* splits have happened and insertion is still unsuccessful.

Case Study – cont'd

city name	hash value: h(city name)
Lianyungang	1101 0111 1011 0101 1110 0101 1110 1111
Nottingham	0010 0110 1101 0101 0111 0010 1100 0101
Cambridge	0110 1001 1000 0011 1100 0010 0101 0000
Venice	0011 0001 0011 0100 1101 1101 0111 0101
Kuala Lumpur	1000 0001 1100 0000 1010 0001 1100 1110
Taiyuan	0010 1001 1000 0011 1100 0010 0101 0000
Java Island	0111 1001 1000 0011 0110 1001 1000 0011
Valencia	1110 1001 1000 0011 1100 0010 0101 0000
Florence	0010 1001 1000 0011 0110 1001 1000 0011
Chiang Mai	0010 1110 0001 0001 0101 0111 1100 1100

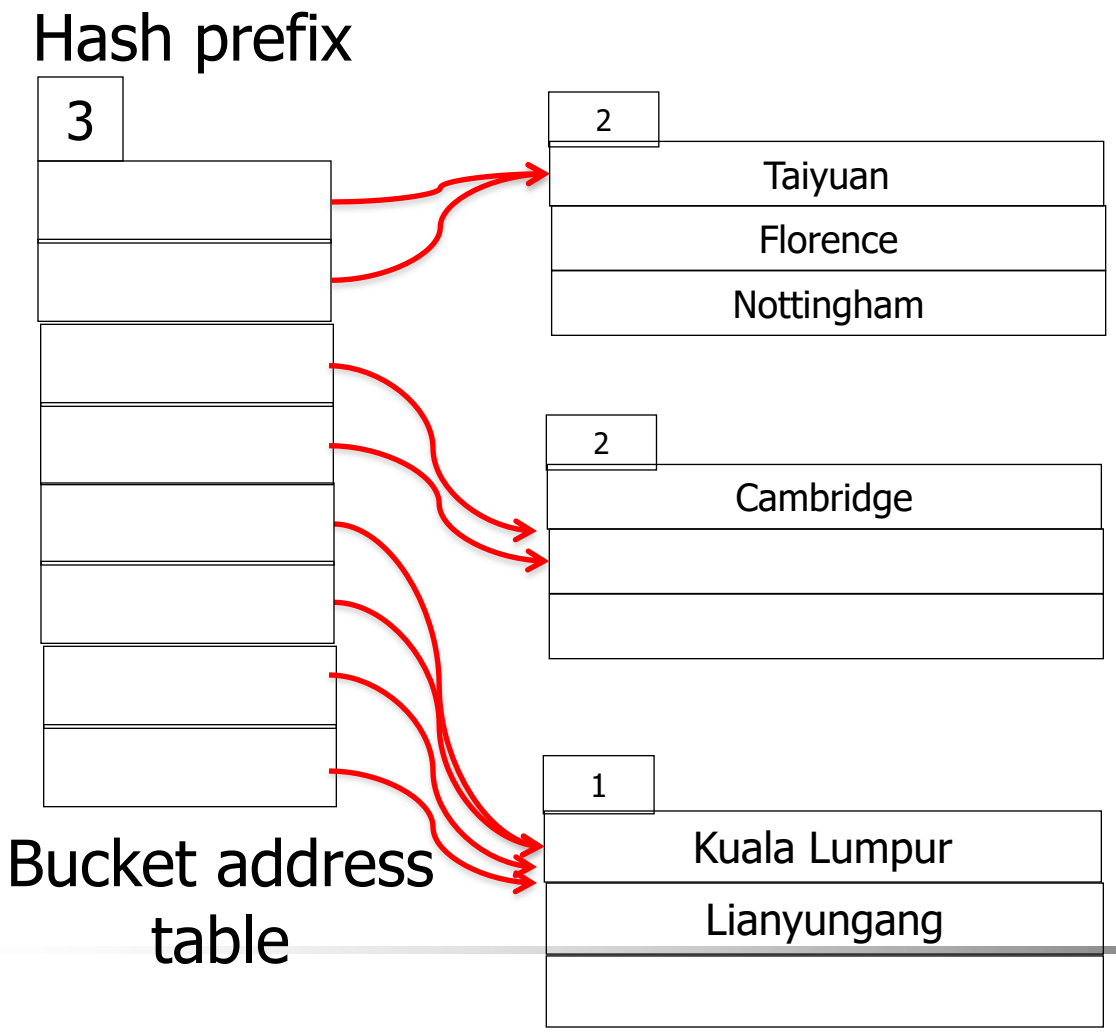
Case Study – cont'd

- Initial extendable hash index
- Task: insert new record with "Taiyuan"



Q2 Solution cont'd

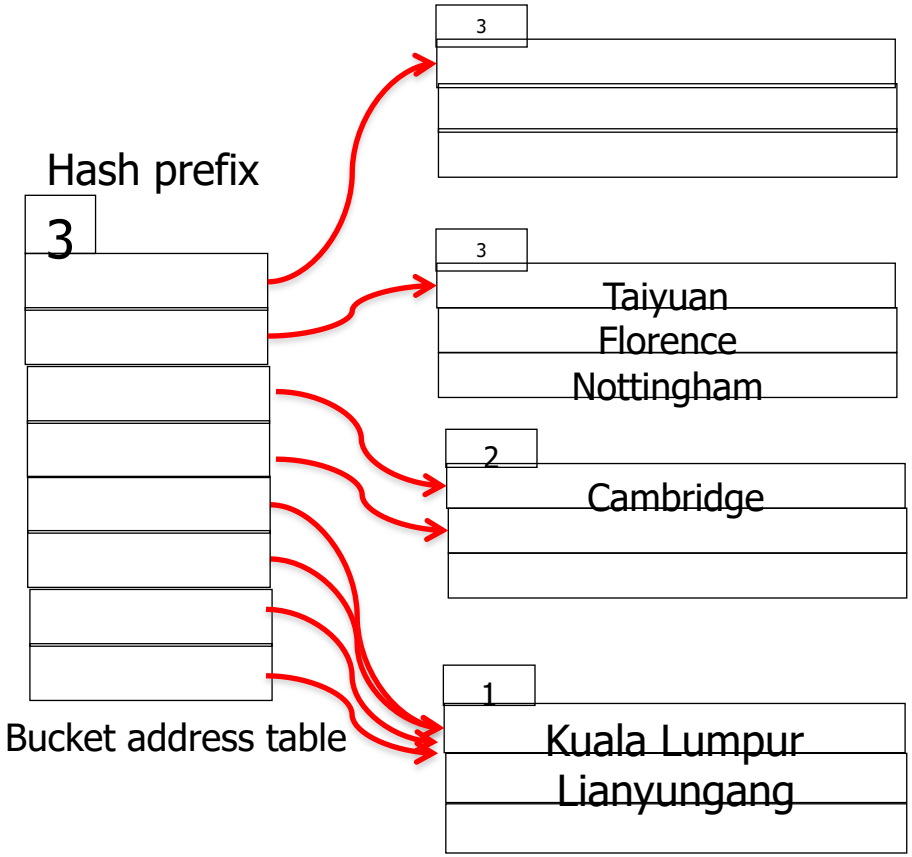
- Intermediate step: insert new record with "Taiyuan"
- Doubling bucket address table



Xi'an Jiaotong-Liverpool University 西交利物浦大學

Q2 Solution cont'd

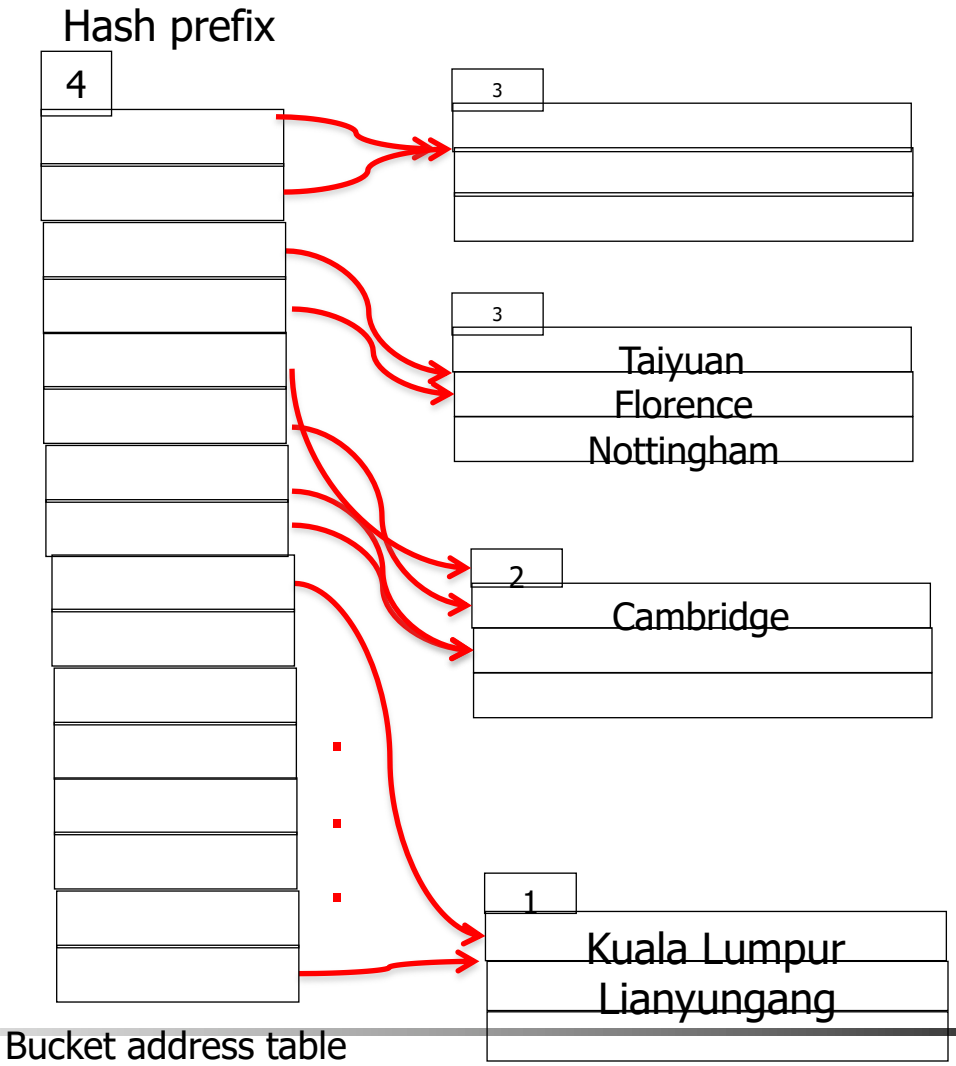
- Intermediate step: insert new record with "Taiyuan"
- Doubling bucket address table
- Split bucket (1st time), reinsert records
- Insert new record unsuccessful



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q2 Solution cont'd

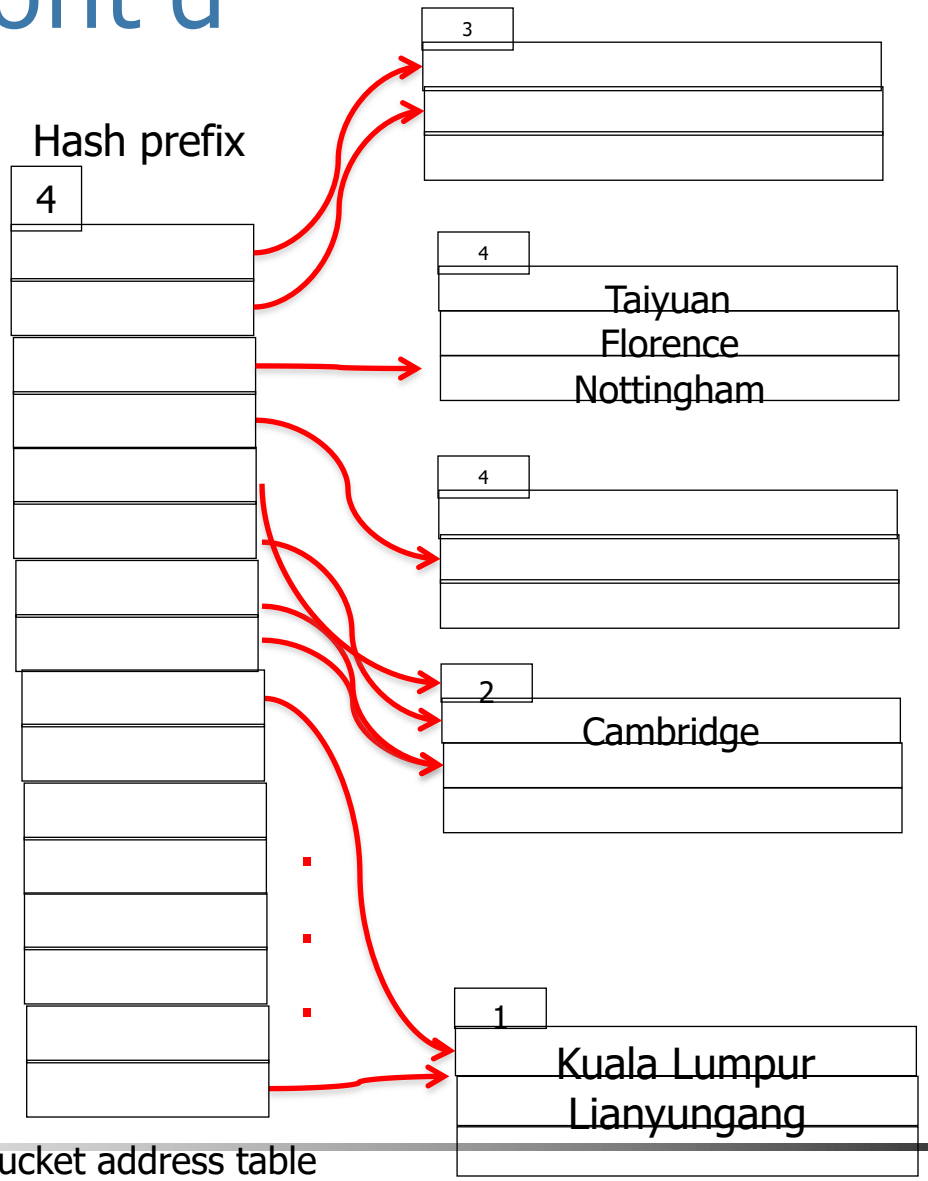
- Intermediate step: insert new record with "Taiyuan"
- Doubling bucket address table
- Split bucket (1st time), reinsert records
- Insert new record unsuccessful
- Doubling bucket address table



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q2 Solution cont'd

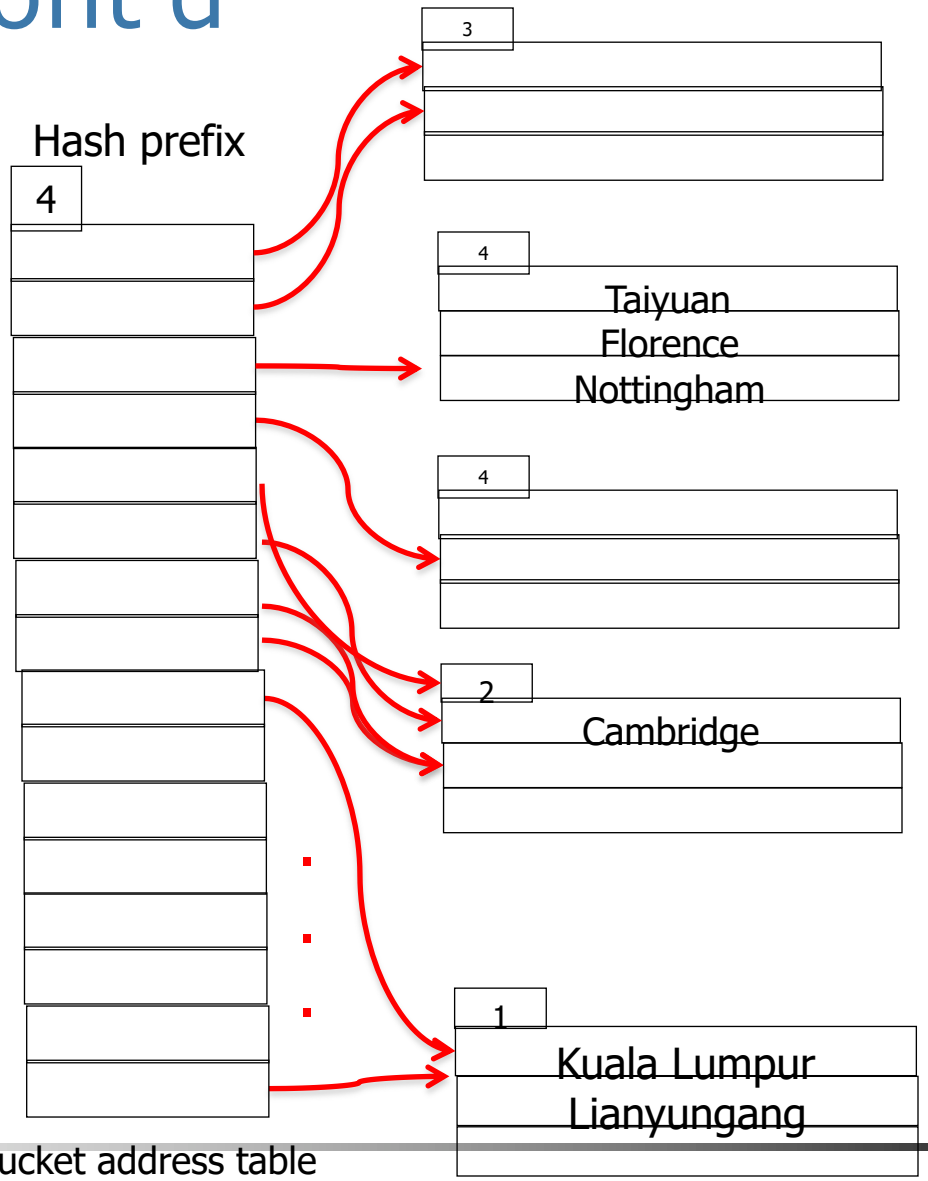
- Intermediate step: insert new record with "Taiyuan"
- Doubling bucket address table
- Split bucket (1st time), reinsert records
- Insert new record unsuccessful
- Doubling bucket address table
- Split bucket (2nd time), reinsert records



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q2 Solution cont'd

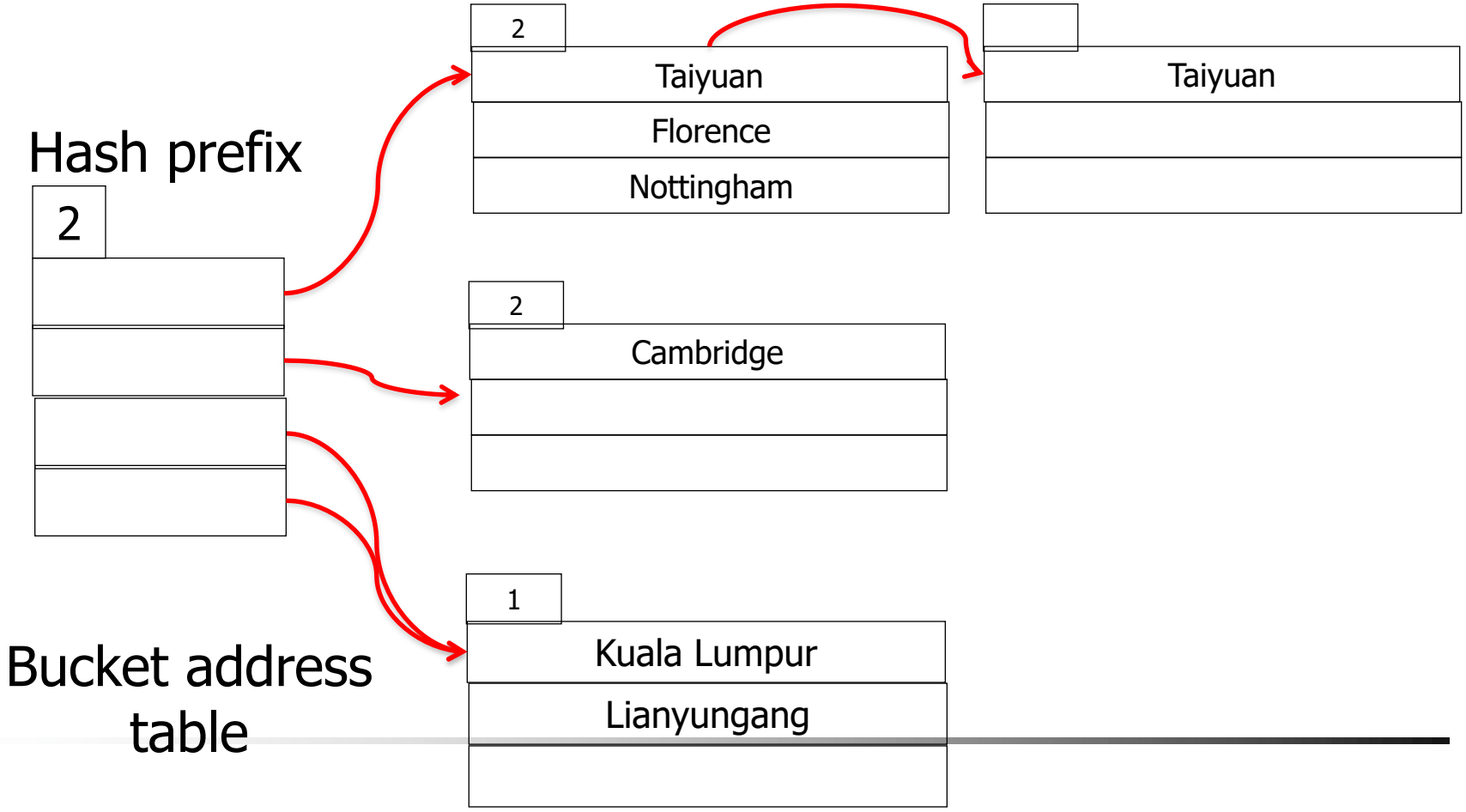
- Intermediate step: insert new record with "Taiyuan"
- Doubling bucket address table
- Split bucket (1st time), reinsert records
- Insert new record unsuccessful
- Doubling bucket address table
- Split bucket (2nd time), reinsert records
- Insert new record unsuccessful
- Two split attempts already!



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q2 Solution cont'd

- *Two attempts but still cannot insert;*
- *Revert to initial index;*
- *Use overflow bucket.*



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Advantages and Disadvantages

- Benefits of extendable hashing
  - Performance does not degrade with growth of file
  - Minimal space overhead
- Disadvantages of extendable hashing
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
    - Cannot allocate very large contiguous areas on disk either
    - Solution: B+-tree structure to locate desired record in bucket address table
  - Changing size of bucket address table is expensive

Choice of Ordered and Hashing Index

- Choice depends on
  - Cost of periodic re-organisation
  - Relative frequency of insertions and deletions
  - Expected type of queries
- In practice:
  - Hash indices are extensively used in-memory (e.g. in hash-join)
  - But not used much on disk
    - Oracle supports static hash file organisation, but not hash index
    - SQL Server and PostgreSQL do not support hashing on disk

Type of Queries and Indices

- Hashing is generally better at retrieving records having specific value.



select A1, A2, … An
from r
where Ai = c

- If range queries are common, ordered indices are preferred



select A1, A2, … An
from r
where Ai ≥ c2 and
Ai ≤ c1

End of Lecture

- Reading
  - Textbook, chapter 24.5